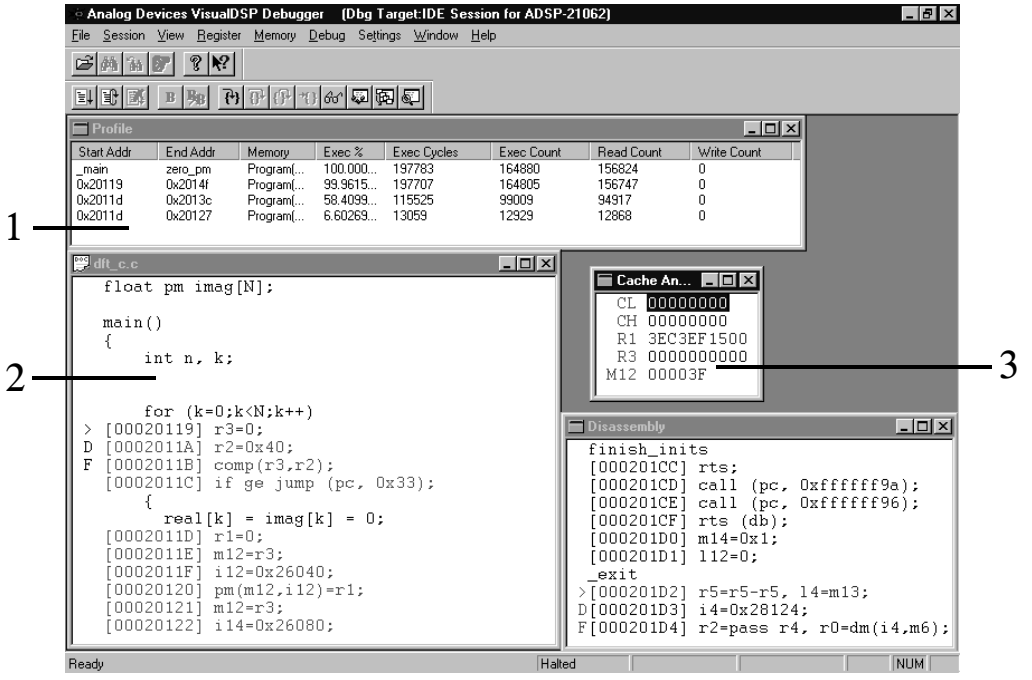

1. Profile code to identify bottlenecks
2. View mixed C and Assembly listings
3. Create custom Register window

FIGURE 29-8
VisualDSP debugging screen. This is a common interface for both simulation and emulation. It can view a C program interspersed with the resulting assembly code, track execution of instructions; examine registers (hardware, software, and memory); trace bus activity; and many other tasks.

edit more than one file at one. This is very convenient, since the final program is created by linking several files together.

Figure 29-8 shows an example screen from the VisualDSP debugger. This is an interface to two different types of tools: simulators and emulators. **Simulators** test the code *within the personal computer*, without even needing a DSP to be present. This is generally the first debugging step after the program is written. The simulator mimics the architecture and operation of the hardware, including: input data streams, interrupts and other I/O. **Emulators** (such as the Analog Devices EZ-ICE®) examine the program operation *on the actual hardware.* This requires the emulator software (on your PC) to be able to monitor the electrical signals *inside* of the DSP. To support this, the SHARC DSPs feature an IEEE 1140.1 JTAG Test Access Port, allowing an external device to track the processor's internal functions.

After you have used an evaluation kit and given some thought to purchasing advanced software tools, you should also consider attending a training class. These are given by many DSP manufacturers. For instance, Analog Devices offers a 3 day class, taught several time a year, at several different locations. These are a great way of learning about DSPs from the experts. Look at the manufacturer's websites for details.